

HANDS ON

Oggi usiamo l'agente vero:
installiamo Claude Code, capiamo come lavora
e scriviamo le prime regole del progetto.

Agenda · mattina

9:00	La demo: l'agente al lavoro, dal vivo
9:20	Cos'era quello che avete visto: il ciclo dell'agente
9:45	Installazione e primo contatto con l'agente
10:45	Pausa, camere spente
11:00	Il primo esercizio vero con l'agente
12:25	Avvio del progetto: il nostro manuale
13:40	Chiusura e diario di bordo

Agenda · pomeriggio

15:00 Lavoro sul manuale, momenti alternati a camere spente

16:10 Pausa, camere spente

16:20 Condivisione e feedback

16:45 Chiusura e diario di bordo

Il problema da risolvere oggi

Nei primi due giorni abbiamo imparato a chiedere meglio e a dare più contesto. Oggi facciamo un passo ulteriore: invece di ricevere soltanto una risposta, chiediamo a un sistema AI di intervenire direttamente su un progetto.

- non basta più “scrivimi un testo”
- serve capire dove l’agente lavora
- serve imparare a dargli regole e confini
- serve verificare quello che produce

DA RISPOSTA A AZIONE

Il punto non è la magia dello strumento. Il punto è imparare a guidare un sistema che può compiere azioni reali sui file.

Obiettivi della giornata

Alla fine di oggi non vogliamo soltanto
“avere installato Claude Code”.

Vogliamo capire che cosa cambia quando un modello AI
può usare strumenti e lavorare dentro una cartella di
progetto.

- definire cos'è un agente AI
- installare e avviare Claude Code
- creare un workspace ordinata
- scrivere AGENTS.md e CLAUDE.md
- iniziare il manuale con un metodo ripetibile

USCIRE AUTONOMI

L'obiettivo minimo è una cartella
funzionante. L'obiettivo vero
è capire come si lavora con un
agente senza farsi trascinare
dal caos.

Che cos'è un agente

Prima di aprire il terminale dobbiamo dare un nome preciso a quello che stiamo per usare.

La domanda giusta

Se un GPT sa già scrivere codice, riscrivere testi, e molto altro... perché serve un agente?

La risposta è semplice: perché scrivere una soluzione e applicarla a un progetto non sono la stessa cosa.

Tra “consigliare” ed “eseguire” c’è un salto enorme.

Che cos'è un Agente: definizione semplice

Un agente AI è un sistema che riceve un obiettivo, osserva il contesto disponibile, sceglie quali strumenti usare, compie azioni e controlla se il risultato ottenuto è coerente con la richiesta.

- non vive solo nella chat
- può usare strumenti esterni
- può lavorare in più passaggi
- può correggersi dopo un errore

OBIETTIVO

+

STRUMENTI

Un agente non è più intelligente “per magia”. È più operativo perché può collegare ragionamento e azione.

Chatbot o agente?

CHATBOT

Un chatbot produce risposte dentro una conversazione. Può spiegare, generare testo, proporre codice o suggerire una procedura. Però, di solito, sei tu a copiare il testo, creare i file, salvare le modifiche e controllare se tutto funziona.

AGENTE

Un agente può entrare in una cartella, leggere i file, modificarli, usare il terminale, lanciare controlli e riportarti il risultato. Non sostituisce il tuo giudizio, ma sposta una parte del lavoro dal “dire” al “fare”.

In una frase: un chatbot propone, un agente fa.

COS'ERA QUELLO CHE AVETE VISTO IERI

Il ciclo dell'agente

Quello che l'agente ha appena fatto segue sempre quattro tempi. Ora che l'avete visto succedere, diamogli un nome.

1

Osserva

Legge la richiesta e capisce la situazione di partenza.

2

Pianifica

Decide come impostare il lavoro, prima di agire.

3

Usa

Agisce davvero: crea il file e ci scrive dentro.

4

Verifica

Controlla il risultato, e se serve lo corregge.

È lo stesso schema per qualsiasi compito, non solo per questo.

Gli strumenti cambiano tutto

La differenza tra un modello che scrive e un agente che lavora sta negli strumenti. Il modello linguistico ragiona e produce istruzioni; gli strumenti gli permettono di agire su file, comandi, ricerche, repository e ambienti di sviluppo.

- filesystem: leggere e scrivere file
- terminale: eseguire comandi
- editor: lavorare sul codice
- Git: vedere differenze e versioni
- test: controllare se qualcosa funziona

TOOLS

Gli strumenti sono le “mani” dell’agente. Senza strumenti resta un consulente; con gli strumenti diventa operativo.

Un esempio concreto

Se chiediamo a un chatbot “scrivi un README”, lui ci restituisce testo. Se chiediamo a Claude Code la stessa cosa dentro una cartella, può creare davvero README.md, organizzarlo, modificarlo dopo un feedback e mostrarci cosa ha cambiato.

- prima: copiare e incollare manualmente
- adesso: lavorare direttamente nel progetto
- prima: risultato isolato
- adesso: risultato dentro una struttura

README.md

La differenza non è il file README. La differenza è che il risultato finisce nel posto giusto, dentro un progetto reale.

Cosa NON è un agente

Un agente non è un collega perfetto, non è un tecnico infallibile e non è un sistema da lasciare libero senza controllo. È un collaboratore digitale potente ma fallibile, che lavora meglio quando riceve obiettivi chiari e limiti espliciti.

- può fraintendere una richiesta
- può modificare più del necessario
- può proporre soluzioni inutilmente complesse
- può avere bisogno di correzioni

NON AUTOPILOTA

La regola è semplice: delegare non significa sparire. Significa assegnare, osservare, correggere e verificare.

Claude Code

Ora diamo un nome allo strumento: Claude Code è un agente che lavora dal terminale, dentro i progetti.

Che cos'è Claude Code

Claude Code porta Claude dentro l'ambiente di lavoro: terminale, cartelle, editor e strumenti di sviluppo. Non serve solo a “fare codice”: può leggere una struttura di progetto, generare documentazione, riorganizzare file e aiutare a verificare il lavoro.

- lavora nella cartella in cui lo avvii
- legge il contesto del progetto
- propone e applica modifiche
- chiede permessi per azioni sensibili
- si usa a cicli, non con un solo prompt

CLAUDE CODE

Per questa lezione lo useremo in modo semplice: creare file, leggere contenuti, riscrivere testi, organizzare un manuale. Il coding arriva dopo.

Perché passa dal terminale

Il terminale è il punto in cui il computer riceve comandi diretti. Claude Code vive lì perché da lì può vedere la cartella corrente, usare strumenti installati, creare file e lanciare controlli.

- non è una chat separata dal computer
- non lavora “nel vuoto”
- ha bisogno di una posizione precisa
- quella posizione è la workspace

TERMINALE

Il terminale non è un luogo “da hacker”. È una stanza di lavoro: se entri nella cartella giusta, Claude lavora lì.

Permessi e responsabilità

Quando un agente può modificare file o eseguire comandi, la sicurezza diventa parte del metodo. Claude Code può chiedere conferme prima di operazioni sensibili: non bisogna approvare in automatico, ma leggere cosa sta per fare.

- controllare la cartella in cui siamo
- leggere i comandi prima di approvare
- fare richieste piccole all'inizio
- salvare copie quando il progetto è importante

CONTROLLA PRIMA

La sicurezza non è paranoia.
È igiene del lavoro: guardo cosa
sta per succedere prima di
premere invio.

Quando usarlo e quando no

Claude Code è utile quando il risultato deve vivere dentro un progetto: file, cartelle, documentazione, codice, note, versioni. È meno adatto quando ci serve soltanto una risposta veloce o un'idea generale.

- usalo per lavorare su materiali reali
- non usarlo per una domanda da trenta secondi
- usalo quando vuoi iterare su file
- non usarlo se non sai dove sta lavorando

SCEGLI LO STRUMENTO

Il punto non è usare sempre l'agente. Il punto è scegliere lo strumento giusto.

Il primo principio operativo

Un agente lavora bene quando il compito è piccolo, concreto e verificabile. Se gli chiediamo “migliora tutto il progetto”, probabilmente farà troppo. Se gli chiediamo “crea un README con titolo, obiettivo e tre sezioni”, possiamo controllare subito il risultato.

- piccolo
- concreto
- verificabile
- correggibile

**PICCOLO
È
MEGLIO**

Le richieste piccole costruiscono fiducia e controllo. Le richieste enormi producono rumore.

Installazione guidata

Installiamo lo strumento senza trasformare il setup in un labirinto.
Se qualcosa non funziona, lo trattiamo come debugging.

Prima: controlliamo il terminale

Apriamo il terminale e verifichiamo di saper eseguire un comando. Non stiamo ancora installando nulla: stiamo solo controllando che il computer risponda.

```
pwd  
ls  
whoami
```

pwd mostra dove siamo; ls mostra cosa c'è nella cartella; whoami mostra l'utente attivo.

Installazione consigliata · macOS / Linux / WSL

Il metodo consigliato attualmente è l'installazione nativa. Su macOS, Linux o WSL si può usare questo comando nel terminale.

```
curl -fsSL https://claude.ai/install.sh | bash
```

Dopo l'installazione, chiudere e riaprire il terminale se il comando claude non viene trovato subito.

Installazione · Windows PowerShell

Su Windows si può usare PowerShell. È importante non confondere PowerShell con CMD: hanno prompt e sintassi diversi.

```
irm https://claude.ai/install.ps1 | iex
```

In PowerShell il prompt di solito inizia con PS. Se appare un errore strano, probabilmente siamo nel terminale sbagliato.

Alternativa npm

Esiste anche l'installazione tramite npm. Oggi però la usiamo solo come alternativa, perché richiede una versione recente di Node e può creare problemi se il sistema è vecchio o configurato male.

```
npm install -g @anthropic-ai/claude-code
```

Se npm dà errori di permessi o Node è vecchio, non insistiamo: usiamo l'installer nativo.

Verifica

Quando l'installazione è finita, controlliamo che il comando sia disponibile. Se non risponde, spesso basta chiudere e riaprire il terminale.

```
claude --version  
claude
```

Il primo comando verifica la versione. Il secondo avvia Claude Code.

Login e accesso

Al primo avvio Claude Code chiede di autenticarsi. L'accesso può dipendere dal tipo di account disponibile: Claude Pro, Max, Team, Enterprise o Console/API. In aula ci interessa arrivare alla schermata in cui Claude Code è pronto dentro una cartella.

- seguire il login guidato
- usare l'account previsto dal corso
- non condividere token o credenziali
- scrivere “fatto” quando si arriva al prompt

LOGIN

Il login è una fase amministrativa, non didattica. L'obiettivo è arrivare alla prima sessione operativa.

Se qualcosa si blocca

Durante un setup è normale incontrare errori. La cosa importante è dare un nome al problema: comando non trovato, permessi, terminale sbagliato, rete, account, versione vecchia.

- copiare l'errore esatto
- non fare dieci tentativi casuali
- dire dove si era arrivati
- chiedere aiuto con messaggio breve

DEBUG CALMO

Il debugging comincia quando smettiamo di dire “non va” e iniziamo a dire “questo comando produce questo errore”.

Workspace: dove lavora l'agente

Un agente deve sapere dove si trova. La cartella non è un dettaglio: è il suo campo d'azione.

Il workspace

Una workspace è la cartella di lavoro del progetto. Dentro ci sono file, istruzioni, appunti e risultati. Quando avviamo Claude Code in una cartella, gli stiamo dicendo: “questo è il posto in cui devi lavorare”.

- una cartella = un contesto operativo
- file ordinati = meno confusione
- istruzioni visibili = comportamento più stabile
- risultati salvati = lavoro riutilizzabile

CARTELLA

=

CONTESTO

Se la cartella è disordinata, anche l'agente parte male. La workspace è una forma di context engineering.

Creiamo la cartella del giorno

Partiamo da una cartella pulita.

Non lavoriamo sul Desktop in modo caotico: creiamo un posto preciso per il laboratorio.

```
cd ~/Desktop  
mkdir Manuale_Day03  
cd Manuale_Day03  
pwd
```

Alla fine pwd deve mostrare la cartella Manuale_Day03.

Avviamo Claude Code nella cartella giusta

Il momento importante è questo: Claude Code va avviato dentro la cartella del progetto, non in una cartella qualsiasi.

```
claude
```

Da qui in poi le richieste riguardano la cartella **Manuale_Day03**.

Prima richiesta sicura

La prima richiesta non deve essere ambiziosa. Chiediamo all'agente di osservare la cartella e descrivere cosa vede.

```
Guarda questa cartella e spiegami cosa contiene.  
Non creare ancora nessun file.
```

Questa richiesta verifica che siamo nella cartella giusta e che Claude Code stia ragionando sul progetto corretto.

Primo file

Ora chiediamo una piccola azione concreta: creare un README iniziale. Il risultato è facile da controllare.

```
Crea un file README.md per questo progetto.  
Deve contenere: titolo, obiettivo, struttura prevista e una nota sul metodo di lavoro.
```

Dopo la creazione apriamo il file e controlliamo se è davvero leggibile.

La sequenza corretta

Quando la sequenza è chiara, molti errori spariscono. La maggior parte dei problemi nasce perché l'agente viene avviato nella cartella sbagliata o riceve richieste troppo grandi troppo presto.

1

Entra nella
cartella

2

Avvia claude

3

Chiedi di osservare

4

Chiedi un file

5

Controlla

Regole persistenti

Ora costruiamo il “manuale dell’agente”:
un file che spiega come deve comportarsi nel progetto.

AGENTS.md: l'idea generale

AGENTS.md è un file Markdown pensato come un README per agenti. Serve a dare istruzioni leggibili, stabili e condivisibili: obiettivo del progetto, struttura, comandi utili, stile, limiti e criteri di verifica.

- è scritto in linguaggio naturale
- vive nella cartella del progetto
- aiuta l'agente a non ripartire da zero
- rende esplicite le regole del lavoro

AGENTS.md

Non è un file magico. È un documento di contesto operativo.

Nota importante: Claude usa CLAUDE.md

Per Claude Code il file nativo di istruzioni persistenti è CLAUDE.md. AGENTS.md è invece un formato più portatile e leggibile anche da altri agenti. Nel nostro laboratorio li trattiamo insieme: AGENTS.md come standard generale, CLAUDE.md come file specifico per Claude Code.

- AGENTS.md = guida portatile per agenti
- CLAUDE.md = istruzioni caricate da Claude Code
- i contenuti possono essere quasi uguali
- l'importante è non confondere il nome del file

AGENTS

+

CLAUDE

Questa slide corregge un'ambiguità importante: non dire che Claude Code legge AGENTS.md automaticamente se non è configurato. Usa CLAUDE.md per Claude.

README.md, AGENTS.md, CLAUDE.md

README.md

È scritto per le persone. Spiega che cos'è il progetto, come leggerlo, come usarlo o come contribuire. Deve essere chiaro anche a chi non userà un agente.

AGENTS.md / CLAUDE.md

Sono scritti per guidare il lavoro dell'agente. Contengono regole operative: come scrivere, cosa non toccare, quali comandi usare, come verificare, quando chiedere conferma.

Creiamo AGENTS.md

Nel progetto creiamo il primo file di regole. Per ora sarà breve: meglio poche regole vere che un documento lungo e generico.

```
touch AGENTS.md  
ls  
# poi apri AGENTS.md nell'editor
```

Se usi VS Code e il comando `code` è disponibile: `code AGENTS.md`

Creiamo anche CLAUDE.md

Poiché oggi usiamo Claude Code, creiamo anche il file che Claude carica come istruzioni di progetto. Possiamo duplicare le regole principali o farle puntare ad AGENTS.md, ma per chiarezza iniziale le teniamo entrambe leggibili.

```
touch CLAUDE.md  
ls
```

Nella pratica del corso: AGENTS.md spiega lo standard; CLAUDE.md rende Claude Code più coerente.

Template minimo per AGENTS.md

Questo è un buon punto di partenza. Non cerca di prevedere tutto: definisce obiettivo, stile, limiti e verifiche.

```
# AGENTS.md

## Obiettivo
Costruire un manuale chiaro su come presentarsi in un nuovo posto di lavoro.

## Stile
Scrivi in italiano semplice, concreto e incoraggiante.

## Regole
- Non cancellare file senza chiedere.
- Mantieni titoli brevi.
- Quando modifichi un file, spiega cosa hai cambiato.

## Verifica
Ogni testo deve essere leggibile da una persona che inizia il suo primo lavoro.
```

Template minimo per CLAUDE.md

CLAUDE.md può contenere le stesse regole, ma rivolte esplicitamente a Claude Code. Deve dire come comportarsi in questa cartella.

```
# CLAUDE.md
```

```
Sei l'agente di supporto per il progetto Manuale_Day03.
```

```
Lavora sempre in italiano.
```

```
Prima di modificare molti file, proponi un piano breve.
```

```
Non cancellare file senza chiedere conferma.
```

```
Quando crei o modifichi un file Markdown, usa titoli chiari, paragrafi brevi e liste leggibili.
```

```
Alla fine di ogni intervento, riassumi cosa hai fatto e cosa resta da fare.
```

Claude Code usa questo file come contesto persistente del progetto.

Regole buone e regole deboli

Una regola utile deve essere concreta, osservabile e collegata al progetto.
Le frasi troppo generiche sembrano rassicuranti, ma guidano poco.

Tipo	Debole	Migliore
Stile	Scrivi bene.	Scrivi in italiano semplice, con frasi brevi e tono incoraggiante.
Limiti	Stai attento.	Non cancellare file e non rinominare cartelle senza chiedere conferma.
Verifica	Controlla.	Dopo ogni modifica, indica file cambiati e motivo della modifica.
Obiettivo	Aiutami.	Aiuta a costruire un manuale per chi inizia un nuovo lavoro.

Cosa non mettere nelle regole

Le regole persistenti non devono diventare un deposito infinito di preferenze casuali. Se scriviamo troppo, l'agente capisce meno. Se scriviamo regole contraddittorie, l'agente oscilla.

- non mettere tutto il programma del corso
- non inserire esempi inutili o vecchi
- non scrivere dieci regole che dicono la stessa cosa
- non mescolare obiettivi, stile e comandi senza ordine

**MENO
MA
MEGLIO**

Un buon file di istruzioni
è breve, vero, aggiornato
e verificabile.

Esercizio guidato

Ora lo facciamo davanti a loro: non una cosa spettacolare,
ma un esercizio controllabile.

ESERCIZIO 1 · osservare prima di agire

Prima di far creare contenuti, chiediamo all'agente di leggere la situazione. Questo impedisce di partire subito con modifiche casuali.

Guarda questa cartella.
Dimmi quali file vedi e spiegami a cosa servono.
Non modificare nulla.

Obiettivo didattico: mostrare la fase Osserva.

ESERCIZIO 2 · creare un file controllabile

Ora gli diamo un compito piccolo: creare la struttura del manuale. Non chiediamo il manuale completo.

```
Crea un file INDICE.md con una proposta di 6 capitoli per il manuale.  
Ogni capitolo deve avere un titolo breve e una frase che spiega a cosa serve.
```

Obiettivo didattico: mostrare una prima azione concreta.

Esercizio 3 · correggere senza ricominciare

La parte importante non è la prima versione. La parte importante è chiedere una modifica precisa sul file già creato.

```
Modifica INDICE.md: rendi i titoli meno scolastici e più vicini a una persona che inizia davvero un lavoro.  
Non cambiare il numero dei capitoli.
```

Obiettivo didattico: mostrare iterazione e controllo.

Cosa osservare durante la demo

Durante la demo non guardiamo solo il risultato finale. Guardiamo come l'agente interpreta il compito, quali file tocca, se chiede permessi, se rispetta le regole e come riassume quello che ha fatto.

- ha lavorato nella cartella giusta?
- ha rispettato AGENTS.md / CLAUDE.md?
- ha modificato solo ciò che doveva?
- il risultato è leggibile?
- ha spiegato le modifiche?

OSSERVARE IL PROCESSO

L'apprendimento sta nel processo, non nel file carino prodotto alla fine.

Quando fermare l'agente

Un errore comune è lasciare l'agente andare avanti troppo a lungo. Se vediamo che ha capito male, sta modificando troppo o sta uscendo dall'obiettivo, lo interrompiamo e lo riportiamo al compito.

- fermalo se cambia file non richiesti
- fermalo se il piano è troppo grande
- fermalo se inventa contesto
- fermalo se ignora le regole

STOP È METODO

Interrompere non è fallire. È guidare.

Esercizi

Ora ogni studente fa lo stesso percorso:
piccolo obiettivo, file reale, modifica, controllo.

Esercizio 1 · crea la workspace

CONSEGNA

Crea una cartella chiamata `Manuale_Day03`, entra nella cartella, avvia Claude Code e chiedigli di osservare il contenuto senza modificare nulla.

✓ FATTO QUANDO

Hai Claude Code avviato nella cartella giusta e sai dire dove sta lavorando.

→ E SE HAI FINITO

Crea una sottocartella appunti e chiedi all'agente di descrivere la struttura.

Esercizio 2 · crea le regole

CONSEGNA

Crea AGENTS.md e CLAUDE.md. Inserisci regole brevi: obiettivo del progetto, tono, limiti, verifica finale. Poi chiedi a Claude Code di leggerle e riassumerle.

✓ FATTO QUANDO

Hai due file di regole leggibili e coerenti.

→ E SE HAI FINITO

Chiedi all'agente di proporti tre miglioramenti alle regole, senza applicarli subito.

Esercizio 3 · primo capitolo

CONSEGNA

Chiedi all'agente di creare capitolo_01.md: un breve capitolo del manuale su "Come presentarsi il primo giorno in un nuovo posto di lavoro"

✓ FATTO QUANDO

Hai un file capitolo_01.md con titolo, introduzione e 3 consigli pratici.

→ E SE HAI FINITO

Chiedi una seconda versione più amichevole, poi confronta le due.

Esercizio 4 · revisione mirata

CONSEGNA

Non chiedere “miglioralo” in modo generico. Scegli un criterio: più breve, più chiaro, meno formale, più concreto, più adatto a chi è alla prima esperienza.

✓ FATTO QUANDO

Hai fatto almeno una revisione mirata sul file esistente.

→ E SE HAI FINITO

Chiedi all’agente di spiegare cosa ha cambiato e perché.

Formula per chiedere aiuto

Quando ti blocchi, non scrivere solo “non funziona”.
Scrivi una mini-scheda del problema.
Così anche io possa aiutarti più velocemente.

- sistema operativo: Mac / Windows / Linux
- cartella in cui ti trovi
- comando lanciato
- errore copiato esattamente
- cosa ti aspettavi succedesse

NON VA

→

DATI

Un errore descritto bene è già
mezzo risolto.

Checklist di fine laboratorio

Prima della pausa o della chiusura controlliamo che ogni studente abbia un risultato minimo. Non deve essere perfetto: deve essere reale, salvato, riapribile e comprensibile.

- cartella Manuale_Day03
- README.md o INDICE.md
- AGENTS.md
- CLAUDE.md
- capitolo_01.md
- una modifica richiesta all'agente dopo la prima versione

RISULTATO REALE

Se manca un file, non è un dramma. Ma bisogna sapere quale passaggio non è stato completato.

Il manuale

Il progetto non è un pretesto:
è il contenitore che rende visibile il lavoro dell'agente.

Perché il manuale funziona come progetto

Un manuale è perfetto per imparare gli agenti perché contiene testo, struttura, revisione, tono, versioni e coerenza. Non richiede subito codice complesso, ma richiede metodo: esattamente quello che vogliamo allenare.

- si costruisce a capitoli
- si può revisionare più volte
- richiede coerenza di tono
- obbliga a usare file reali
- permette feedback tra pari

MANUALE

=

PROGETTO

Il manuale è “soft”, ma
tecnicamente è un progetto:
cartelle, file, regole, versioni.

Struttura consigliata

Per evitare confusione, diamo al progetto una struttura semplice e stabile.

```
Manuale_Day03/  
├─ README.md  
├─ AGENTS.md  
├─ CLAUDE.md  
├─ INDICE.md  
├─ capitolo_01.md  
└─ appunti/  
    └─ diario_bordo.md
```

Non serve una struttura complessa: serve una struttura che tutti possano capire.

Prompt per avviare il manuale

Questo prompt dà all'agente un obiettivo chiaro, un formato e un criterio di qualità.

```
Crea INDICE.md per un manuale intitolato "Come presentarsi in un nuovo posto di lavoro".  
Scrivi 6 capitoli.  
Per ogni capitolo inserisci: titolo, scopo, 3 domande guida.  
Usa tono semplice, concreto e rispettoso.
```

Dopo la generazione leggiamo se i capitoli sono davvero utili.

Prompt per un capitolo

Quando generiamo un capitolo, chiediamo una struttura ripetibile. Così il manuale non diventa una somma di testi casuali.

```
Crea capitolo_01.md partendo dal primo capitolo di INDICE.md.  
Struttura: titolo, breve introduzione, 5 consigli pratici, 3 errori da evitare, una mini-checklist finale.  
Scrivi per una persona alla prima esperienza di lavoro.
```

La struttura rende il testo più facile da revisionare.

Revisione tra pari

Ogni studente può leggere il file di un compagno e indicare una modifica precisa. Poi l'autore chiede all'agente di applicare quella modifica. In questo modo il feedback umano entra nel ciclo dell'agente.

- una persona legge
- una persona dà un criterio
- l'autore formula il prompt
- l'agente modifica il file
- si controlla il risultato

UMANO NEL CICLO

L'agente non sostituisce il confronto. Lo rende più rapido e visibile.

Chiusura

Raccogliamo quello che abbiamo imparato e prepariamo il salto del Day 04.

Cosa portiamo a casa

Oggi abbiamo fatto il passaggio centrale del corso: dall'AI che risponde all'AI che opera dentro un progetto. Abbiamo visto che la potenza dell'agente dipende da tre cose: obiettivo chiaro, cartella giusta, regole scritte bene.

- un agente usa strumenti
- Claude Code lavora nella workspace
- AGENTS.md e CLAUDE.md guidano il comportamento
- le richieste piccole sono più controllabili
- la verifica resta responsabilità nostra

DAL DIRE AL FARE

Sintesi concettuale. Non solo
“abbiamo installato”.

Diario di bordo

Prima di chiudere, ogni studente scrive tre righe nel `diario_bordo.md`. Non serve un testo lungo: serve fissare cosa è successo e cosa resta da chiarire.

- una cosa che ho capito
- una cosa che mi ha bloccato
- una cosa che voglio provare meglio domani

3 RIGHE

Il diario di bordo è memoria umana del processo. Aiuta anche te a capire dove ripartire.

Domani

Nel Day 04 renderemo il lavoro più stabile: useremo meglio la documentazione, impareremo a mantenere il contesto nel tempo e capiremo come evitare che l'agente accumuli confusione.

- context engineering applicato ai file
- documentazione come memoria del progetto
- regole più precise
- revisioni e versioni
- primi workflow ripetibili

DAY 04

Domani non aggiungiamo magia.
Aggiungiamo struttura.

Scheda prompt rapida

Questi prompt sono abbastanza piccoli da funzionare bene in laboratorio.

1. Guarda questa cartella e spiegami cosa contiene. Non modificare nulla.
2. Crea README.md con titolo, obiettivo e struttura del progetto.
3. Modifica il file rendendolo più chiaro, ma non cambiare la struttura.
4. Riassumi cosa hai cambiato e quali file hai toccato.

Prompt pronti da copiare.